

# SOFTWARE CHANGE IMPLEMENTATION REPORT

## CR-004: Student Input Validation

| Field                 | Details                  |
|-----------------------|--------------------------|
| Change Request ID     | CR-004                   |
| Change Request Title  | Student Input Validation |
| Student Name          | [Student Name]           |
| Student ID            | [Student ID]             |
| Implementation Status | Completed                |
| Date                  | 8 October 2026           |

## 1. Change Request Summary

### 1.1 Business Requirement

The Student Management System is an administrative web application used by ABC Institute to manage student records. Previously, the student registration and edit forms accepted any input without client-side or server-side format verification, allowing invalid, incomplete, or arbitrary data (such as non-alphabetic names, arbitrary string lengths, malformed emails, and invalid phone numbers) to be submitted and stored in the database. CR-004 was requested to implement robust client-side and server-side input validation across all student data attributes to ensure data integrity, system security, and an improved administrator user experience.

### 1.2 Existing Behaviour

Before implementation:

- The student Add and Edit modal forms accepted any input without client-side format or length verification.
- Form submission was permitted with empty fields or invalid data formats.
- No inline error feedback, visual indicators, or field guidance was displayed to administrators.
- Server-side endpoints `POST /api/students/` and `PUT /api/students/:id` accepted unvalidated payloads directly into database queries.
- Passwords of arbitrary weakness or length (even 1 character) were accepted upon student creation.
- Existing Edit and Delete operations used `student_id`.

## 1.3 Requested Change

Comprehensive client-side and server-side input validation was required for the student Add and Edit forms. The validation must:

- Validate all required student fields before form submission.
- Require a valid registration number (alphanumeric characters, hyphens, and underscores only).
- Require first name and last name to contain at least 3 alphabetic characters only.
- Validate that the email address follows a standard email format.
- Require phone number to contain exactly 10 numeric digits.
- Enforce a strong password policy (minimum 8 characters, uppercase, lowercase, digit, and special character) when creating a new student.
- Display clear, immediate per-field validation error messages to administrators.
- Apply server-side validation to reject invalid direct API requests with HTTP 400 Bad Request.
- Preserve all existing Add, Edit, Delete, authentication, and authorization functionality.

## 2. Implementation

### 2.1 Implementation Summary

The CR-004 requirements were implemented across both the backend and frontend of the Student Management System. A dedicated validation utility was introduced in the backend to validate all student attributes and enforce format and security constraints. Server-side validation was integrated into both `addStudent` and `editStudent` controller handlers, ensuring that requests bypassing the client interface are immediately rejected with HTTP 400 Bad Request and structured error messages.

On the frontend, real-time client-side validation was implemented in `StudentFormModal.jsx`. Each field validates on user blur and input change, displaying clear inline error messages beneath affected inputs. Password policy validation is dynamically enforced during student creation and safely bypassed during student updates. Existing CRUD operations, database schema, and administrator authorization remain fully preserved.

### 2.2 Affected Files

| File                                         | Change   | Purpose                                                                                                                                                                        |
|----------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| backend/src/utils/validation.js              | New      | Centralised validation utility with individual field validators and <code>validateStudentData()</code>                                                                         |
| backend/src/controllers/studentController.js | Modified | Integrated <code>validateStudentData()</code> into <code>addStudent</code> and <code>editStudent</code> ; added <code>ER_DUP_ENTRY</code> handling in <code>editStudent</code> |
| frontend/src/components/StudentFormModal.jsx | Modified | Implemented client-side field validation, inline error state ( <code>fieldErrors</code> , <code>touched</code> ), blur handlers, and password strength checks                  |
| frontend/src/styles/dashboard.css            | Modified | Added styles for <code>.field-error</code> , <code>.field-error-msg</code> , and <code>.field-hint</code>                                                                      |

**Additional files:** None. No database schema changes, no new API endpoints, and no external dependencies were introduced.

### 2.3 Technical Changes

#### Backend Validation Utility

A new validation module was implemented in `backend/src/utils/validation.js` providing dedicated validator functions for registration number, first name, last name, email, phone number, and password, as well as a composite `validateStudentData(data,`

`isEdit`) function. Regular expressions enforce strict character whitelists and structural formats.

### Server-Side Integration

In `backend/src/controllers/studentController.js`, `validateStudentData()` is invoked at the very beginning of both `addStudent` and `editStudent`. If validation fails, execution halts immediately and returns HTTP 400 Bad Request with a structured errors object.

### Client-Side Inline Validation

In `frontend/src/components/StudentFormModal.jsx`, field-level validation mirrors the server-side rules. State hooks track `fieldErrors` and `touched` status. Inline error text and red highlights appear on blur and clear as the user types.

### Strong Password Policy

The password validation enforces security standards during new student creation:

- Minimum 8 characters in length.
- At least one uppercase letter (A–Z).
- At least one lowercase letter (a–z).
- At least one numeric digit (0–9).
- At least one special character (e.g. `!`, `@`, `#`, `$`, `%`, `^`, `&`, `*`).

Passwords continue to be securely hashed using `bcryptjs` before storage. Password validation is bypassed during student editing.

### Existing CRUD Preservation

- Add, Edit, and Delete student operations continue to function as expected.
- Administrator authentication (JWT) and role authorization (ADMIN) remain unchanged.
- Edit and Delete operations continue to use the unique `student.student_id`.
- Existing API routes remain unchanged; no database migrations were required.

## 2.4 Implementation Logic

1. Administrator opens the Add Student or Edit Student modal form.
2. Administrator enters input into student fields (registration number, names, email, phone, password).
3. On field blur, client-side validation evaluates the field against specific regex and length constraints.
4. If invalid, an inline error indicator and red border are displayed immediately under the input.
5. On submit, all fields are validated simultaneously; submission is blocked if any validation error exists.
6. If client-side validation passes, form data is submitted via API request.
7. Backend controller receives request and immediately executes `validateStudentData()`.

8. If any server-side validation rule fails, HTTP 400 Bad Request is returned with structured errors.
9. If valid, the database operation (INSERT or UPDATE) proceeds normally.
10. Student list refreshes and updated records are displayed on the administrator dashboard.

## **2.5 Error and Data Handling**

The implementation handles edge cases and data validation errors safely:

- Missing or null values are caught by required-field checks before regex evaluation.
- Phone numbers are strictly validated as 10-digit strings to prevent non-numeric values and incorrect lengths.
- Duplicate registration numbers or emails trigger HTTP 409 Conflict with descriptive messages.
- Existing error alerts on the modal interface are retained to display backend error responses clearly.

## 3. Testing & Verification

### 3.1 Testing Approach

The implementation was verified through comprehensive manual testing and automated verification across both the frontend user interface and backend API endpoints.

Testing Types:

- Functional testing
- Positive testing
- Negative / validation testing
- Boundary testing
- Security / bypass testing
- UI testing
- Regression testing
- CRUD verification

Environment: Development

Browser: Chrome

Test Role: Administrator

### 3.2 Test Cases

Client-Side Validation Test Cases:

| Test ID | Test Case                                         | Expected Result                                   | Actual Result                  | Status |
|---------|---------------------------------------------------|---------------------------------------------------|--------------------------------|--------|
| TC-01   | Submit form with all fields empty                 | Error shown for each required field; form blocked | Errors displayed; form blocked | Pass   |
| TC-02   | Enter registration number with special characters | Validation error shown                            | Error displayed                | Pass   |
| TC-03   | Enter first name < 3 characters                   | Error: First name must be at least 3 characters   | Error displayed                | Pass   |
| TC-04   | Enter first name with exactly 3 characters        | Accepted; no error                                | Accepted                       | Pass   |
| TC-05   | Enter first name with numeric / special chars     | Error: First name must contain letters only       | Error displayed                | Pass   |
| TC-06   | Enter last name < 3 characters                    | Error: Last name must be at least 3 characters    | Error displayed                | Pass   |
| TC-07   | Enter invalid email (no @ or domain)              | Error: Please enter a valid email address         | Error displayed                | Pass   |
| TC-08   | Enter valid email address                         | Accepted; no error                                | Accepted                       | Pass   |
| TC-09   | Enter phone number with non-numeric chars         | Error: Phone number must contain digits only      | Error displayed                | Pass   |

|       |                                              |                                               |                         |      |
|-------|----------------------------------------------|-----------------------------------------------|-------------------------|------|
| TC-10 | Enter phone number < 10 digits               | Error: Phone number must be exactly 10 digits | Error displayed         | Pass |
| TC-11 | Enter phone number > 10 digits               | Error: Phone number must be exactly 10 digits | Error displayed         | Pass |
| TC-12 | Enter valid 10-digit phone number            | Accepted; no error                            | Accepted                | Pass |
| TC-13 | Password < 8 characters                      | Error: Password must be at least 8 characters | Error displayed         | Pass |
| TC-14 | Password without uppercase letter            | Error: Must include an uppercase letter       | Error displayed         | Pass |
| TC-15 | Password without lowercase letter            | Error: Must include a lowercase letter        | Error displayed         | Pass |
| TC-16 | Password without digit                       | Error: Must include a number                  | Error displayed         | Pass |
| TC-17 | Password without special character           | Error: Must include a special character       | Error displayed         | Pass |
| TC-18 | Password meeting all requirements            | Accepted; no error                            | Accepted                | Pass |
| TC-19 | Submit Add Student with all valid data       | Student created; appears in table             | Student created         | Pass |
| TC-20 | Edit student with valid updated data         | Student updated; changes reflected            | Student updated         | Pass |
| TC-21 | Edit student without touching password       | Password not required; update succeeds        | Update succeeds         | Pass |
| TC-22 | Edit/Delete correct student using student_id | Correct record affected                       | Correct record affected | Pass |

### Server-Side (API) Test Cases:

| Test ID | Test Case                                             | Expected Result                        | Actual Result     | Status |
|---------|-------------------------------------------------------|----------------------------------------|-------------------|--------|
| TC-S01  | POST /api/students/ with empty firstName              | HTTP 400 with validation error message | HTTP 400 returned | Pass   |
| TC-S02  | POST /api/students/ with firstName = "Jo" (< 3 chars) | HTTP 400 with firstName error          | HTTP 400 returned | Pass   |
| TC-S03  | POST /api/students/ with numeric firstName            | HTTP 400 with firstName error          | HTTP 400 returned | Pass   |
| TC-S04  | POST /api/students/ with invalid email format         | HTTP 400 with email error              | HTTP 400 returned | Pass   |
| TC-S05  | POST /api/students/ with phone < 10 digits            | HTTP 400 with phone error              | HTTP 400 returned | Pass   |
| TC-S06  | POST /api/students/ with non-numeric phone            | HTTP 400 with phone error              | HTTP 400 returned | Pass   |
| TC-S07  | POST /api/students/ with weak password                | HTTP 400 with password error           | HTTP 400 returned | Pass   |

|        |                                           |                                |                   |      |
|--------|-------------------------------------------|--------------------------------|-------------------|------|
| TC-S08 | POST /api/students/ with all valid data   | HTTP 201; student created      | HTTP 201 returned | Pass |
| TC-S09 | PUT /api/students/:id with invalid email  | HTTP 400 with validation error | HTTP 400 returned | Pass |
| TC-S10 | PUT /api/students/:id with all valid data | HTTP 200; student updated      | HTTP 200 returned | Pass |

### 3.3 Test Summary

| Test Set               | Total Cases | Passed | Failed | Pass Rate |
|------------------------|-------------|--------|--------|-----------|
| Client-Side Validation | 22          | 22     | 0      | 100%      |
| Server-Side / API      | 10          | 10     | 0      | 100%      |
| Overall                | 32          | 32     | 0      | 100%      |

**Overall Test Result: Pass**



## 4. Acceptance Criteria Verification

| ID     | Acceptance Criterion                                                            | Verification Method   | Status |
|--------|---------------------------------------------------------------------------------|-----------------------|--------|
| AC-001 | Administrators cannot submit the form when required fields are empty.           | TC-01                 | Met    |
| AC-002 | First names containing fewer than 3 characters are rejected.                    | TC-03, TC-S02         | Met    |
| AC-003 | Last names containing fewer than 3 characters are rejected.                     | TC-06                 | Met    |
| AC-004 | Names containing invalid characters are rejected.                               | TC-05, TC-S03         | Met    |
| AC-005 | Invalid email addresses are rejected.                                           | TC-07, TC-S04         | Met    |
| AC-006 | Phone numbers with fewer than 10 digits are rejected.                           | TC-10, TC-S05         | Met    |
| AC-007 | Phone numbers with more than 10 digits are rejected.                            | TC-11                 | Met    |
| AC-008 | Phone numbers with non-numeric characters are rejected.                         | TC-09, TC-S06         | Met    |
| AC-009 | A valid 10-digit phone number is accepted.                                      | TC-12                 | Met    |
| AC-010 | Weak passwords are rejected when creating a new student.                        | TC-13 – TC-17, TC-S07 | Met    |
| AC-011 | A strong valid password is accepted.                                            | TC-18, TC-S08         | Met    |
| AC-012 | A valid student can be successfully created.                                    | TC-19, TC-S08         | Met    |
| AC-013 | Invalid student information cannot be successfully created.                     | TC-01 – TC-17         | Met    |
| AC-014 | Invalid student information cannot be successfully updated.                     | TC-S09                | Met    |
| AC-015 | A valid student can be successfully updated.                                    | TC-20, TC-S10         | Met    |
| AC-016 | Direct API requests with invalid data are rejected by the backend.              | TC-S01 – TC-S09       | Met    |
| AC-017 | Existing Edit and Delete functionality continues to affect the correct student. | TC-22                 | Met    |
| AC-018 | Existing administrator authentication and authorization continue to work.       | Regression testing    | Met    |

## 5. Evidence

The following screenshots provide evidence of the implemented validation functionality. Evidence items are identified and referenced to the relevant acceptance criteria.

### Evidence 01 – Client-Side Inline Field Validation Errors

Related: AC-001, AC-002, AC-003, AC-004, AC-005, AC-006, AC-007, AC-008, AC-010

The screenshot shows a modal form titled "Add New Student" with a close button (X). Below the title is a subtitle "Create a new student account." A red warning box at the top states: "Please fix the highlighted errors before submitting." The form contains six input fields arranged in three rows and two columns. Each field has a red border and a red warning icon (triangle) with an error message below it: "First name is required." for the First Name field, "Last name is required." for the Last Name field, "Email address is required." for the Email field, "Phone number is required." for the Phone Number field, and "Password is required." for the Password field. The Registration Number field contains the text "dfgdf333" and has a hint "Letters, numbers, hyphens, or underscores only." at the bottom. At the bottom of the form are two buttons: "Cancel" and "Create Student".

Figure 1: Student Add/Edit form showing per-field inline error messages beneath invalid inputs with red border highlight and warning icons.

### Evidence 02 – Successful Student Creation with Valid Data

Related: AC-011, AC-012

The screenshot shows the "Students" administrator dashboard. At the top, there is a title "Students" and a subtitle "Manage student accounts and profiles." A blue button "+ Add Student" is in the top right. Below the subtitle is a search bar with the placeholder text "Search by registration number, name, or email...". Under the search bar, it says "3 students". Below this is a table with the following data:

| REGISTRATION NO. | NAME           | EMAIL             | PHONE      | ACTIONS                                     |
|------------------|----------------|-------------------|------------|---------------------------------------------|
| dfgdf333         | gaad addsa     | abc@gmail.com     | 1012323664 | <a href="#">Edit</a> <a href="#">Delete</a> |
| REG002           | Sarah Fernando | sarah@example.com | 1421111111 | <a href="#">Edit</a> <a href="#">Delete</a> |
| REG001           | John Perera    | john@example.com  | 0712345678 | <a href="#">Edit</a> <a href="#">Delete</a> |

Figure 2: Student Add form submitted with valid data — student appears in the administrator dashboard table after successful creation.

## Evidence 03 – Successful Student Update with Valid Data

Related: AC-015

**Students**  
Manage student accounts and profiles.

Search by registration number, name, or email

3 students

| REGISTRATION NO. | NAME       | EMAIL          | PHONE      | ACTIONS                                     |
|------------------|------------|----------------|------------|---------------------------------------------|
| dfgdf333         | htte hrree | htte@gmail.com | 1012323664 | <a href="#">Edit</a> <a href="#">Delete</a> |
| REG002           | Sa         |                |            | <a href="#">Edit</a> <a href="#">Delete</a> |
| REG001           | Jo         |                |            | <a href="#">Edit</a> <a href="#">Delete</a> |

**Edit Student**  
Update student information.

**Registration Number**  
dfgdf333  
Letters, numbers, hyphens, or underscores only.

**First Name**  
htte  
Min 3 characters, letters only.

**Last Name**  
hrree  
Min 3 characters, letters only.

**Email**  
htte@gmail.com  
e.g. student@example.com

**Phone Number**  
1012323664  
Exactly 10 digits, numbers only.

[Cancel](#) [Update Student](#)

Figure 3: Student Edit form submitted with valid data — updated values reflected in the administrator dashboard table.

| REGISTRATION NO. | NAME       | EMAIL          | PHONE      | ACTIONS                                     |
|------------------|------------|----------------|------------|---------------------------------------------|
| dfgdf333         | htte hrree | htte@gmail.com | 1012323664 | <a href="#">Edit</a> <a href="#">Delete</a> |

## Evidence 04 – Backend Rejection of Invalid API Request

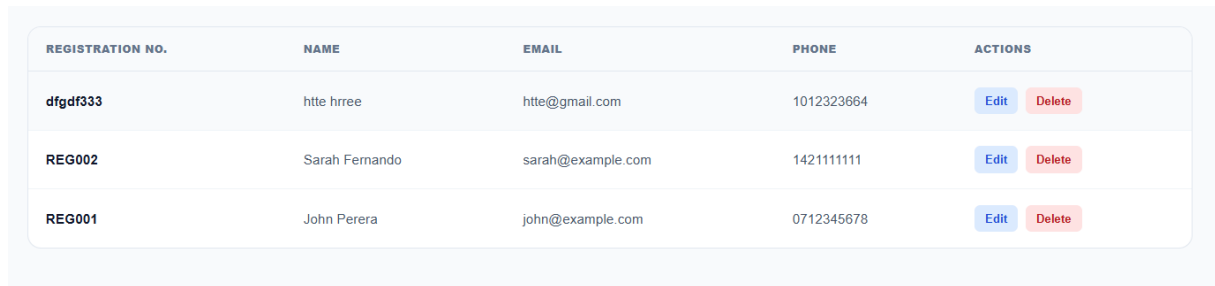
Related: AC-016

```
>>
STATUS CODE: 400
{
  "success": false,
  "message": "First name must contain at least 3 characters",
  "errors": {
    "firstName": "First name must contain at least 3 characters",
    "lastName": "Last name must contain valid alphabetic characters only",
    "email": "Email must follow a valid email format",
    "phoneNumber": "Phone number must contain exactly 10 digits",
    "password": "Password must contain at least 8 characters"
  }
}
PS C:\Users\USER\Desktop\JF.ICT.24.16\student-management-system>
```

Figure 4: Direct invalid API request returning HTTP 400 Bad Request with structured validation errors in terminal / API client.

## Evidence 05 – Existing Edit and Delete Functionality

Related: AC-017, AC-018



The screenshot displays an administrator dashboard with a table containing three rows of user registration data. Each row includes a registration number, name, email, phone number, and two action buttons: 'Edit' (blue) and 'Delete' (red). The table is set against a light gray background with a white border.

| REGISTRATION NO. | NAME           | EMAIL             | PHONE      | ACTIONS                                     |
|------------------|----------------|-------------------|------------|---------------------------------------------|
| dfgdf333         | htte hrree     | htte@gmail.com    | 1012323664 | <a href="#">Edit</a> <a href="#">Delete</a> |
| REG002           | Sarah Fernando | sarah@example.com | 1421111111 | <a href="#">Edit</a> <a href="#">Delete</a> |
| REG001           | John Perera    | john@example.com  | 0712345678 | <a href="#">Edit</a> <a href="#">Delete</a> |

Figure 5: Administrator dashboard showing Edit and Delete operations continuing to function correctly after validation changes.

## 6. Final Result

| Item                | Result    |
|---------------------|-----------|
| Requirements        | Met       |
| Implementation      | Completed |
| Testing             | Passed    |
| Acceptance Criteria | All Met   |
| Existing CRUD       | Verified  |
| Final Status        | Accepted  |

### 6.1 Final Summary

CR-004 was implemented as a comprehensive input validation enhancement for the Student Management System. Client-side inline validation provides immediate feedback to administrators on blur and change in StudentFormModal.jsx, ensuring that registration numbers, names, emails, phone numbers, and strong passwords adhere strictly to defined formats. Server-side validation in backend/src/utlis/validation.js and studentController.js guarantees that direct API requests are verified and invalid data is rejected with HTTP 400 Bad Request. All 18 acceptance criteria were verified and passed, and existing student CRUD operations and administrative authorization remain completely functional.

### 6.2 Issues / Limitations

The existing seed data contains an initial phone number stored as "0" for student REG002 prior to the introduction of validation rules. When an administrator edits this specific legacy record, the form correctly prompts the administrator to provide a valid 10-digit phone number in compliance with the new validation policy.

No significant blocking issues identified.

Student Name: \_\_\_\_\_

Student ID: \_\_\_\_\_

Date: 8 October 2026

Signature: \_\_\_\_\_

~ END OF REPORT ~